

English Conversation Memorization Site

Hardening Report

Date: 2026-06-16 | Branch: codex/project-hardening-docs | Preset: standard_business_brief

Executive Summary

This pass kept the project architecture intentionally small: static HTML, CSS, JavaScript, and data.json served by GitHub Pages. The only code hardening change replaces startup data-load failure innerHTML interpolation with DOM nodes and textContent, then bumps the script query key from script.js?v=21 to script.js?v=22.

Verification passed locally with Node tests, JavaScript syntax check, whitespace diff check, and a real Microsoft Edge browser flow against http://127.0.0.1:8022/.

Project Purpose

The site is a personal memorization helper for English conversation and basic verb practice. It lets the learner choose a course, select a day, reveal answers, replay Korean or English TTS, switch ordering modes, and store review progress in browser localStorage.

Runtime Architecture

```
flowchart LR
    Browser["Browser UI"] --> Index["index.html"]
    Index --> Script["script.js?v=22"]
    Index --> Styles["style.css?v=9"]
    Script --> Data["data.json?v=10"]
    Script --> Storage["localStorage"]
    Script --> Speech["Web Speech API"]
    Data --> CourseData["CourseData[\"Conversation and Basic Verbs courses\"]"]
    Storage --> Preferences["Preferences[\"TTS, theme, font, review stats\"]"]
```

From	To	Purpose
Browser UI	index.html	Static page shell and controls.
index.html	script.js?v=22	Cache-busted runtime logic.
script.js	data.json?v=10	Public course and card data.
script.js	localStorage	User-only preferences and review history.
script.js	Web Speech API	Browser-native TTS playback.

Module Map

File	Role
index.html	Static document structure, controls, modals, and cache-busted script/style references.
style.css	Responsive layout, dark mode, modal, card, controls, and stats table styling.
script.js	Data loading, course/day normalization, rendering, navigation, TTS, theme/font settings, review stats, and refresh cache busting.
data.json	Public course data for conversation and basic verbs lessons.

File	Role
tests/*.test.js	Node VM tests for normalization, rendering, TTS behavior, storage resilience, and course data integrity.

Key Flows

- Page load calls `fetch(buildDataUrl())`, where `DATA_VERSION` remains 10.
- A refresh query parameter on the page is copied into the `data.json` URL so the Refresh button busts the GitHub Pages JSON cache.
- QuizApp normalizes course data once per course and renders card text with `textContent`.
- TTS calls stay in browser APIs and store voice/rate settings locally.
- Review stats are namespaced by course in `localStorage` as `reviewStats:`.
- If `data.json` fails to load, `renderLoadFailure()` creates DOM nodes and writes the error detail via `textContent`.

Hardening Change

Status	Area	Evidence	Resolution
Fixed	Startup error rendering	HEAD used <code>container.innerHTML</code> with <code>\${error.message}</code> in the load failure path.	Added <code>renderLoadFailure()</code> and regression coverage so error details are written as text nodes.
Verified safe	Remaining <code>innerHTML</code> use	Current source has only <code>element.innerHTML = ""</code> in <code>replaceElementChildren()</code> fallback.	It clears children and does not interpolate external data. Modern browsers use <code>replaceChildren()</code> .
Verified safe	Stats table rendering	<code>renderStats()</code> creates cells and assigns <code>textContent</code> .	Malformed <code>localStorage</code> stats are normalized by tests.
Verified safe	Refresh token handling	<code>buildDataUrl()</code> encodes the page refresh parameter before appending it to <code>data.json</code> .	Existing cache-busting behavior preserved.

Security Review

Mode: daily, zero-noise review. No remaining high-confidence vulnerabilities were found after the startup failure rendering fix. This is not a substitute for a professional security audit.

Surface	Result
Public endpoints	Static GitHub Pages files only.
Auth/session	None. No server-side account or privileged flow.
Secrets	No concrete key-format matches in source scan. <code>.env</code> is ignored, and no <code>.env</code> history was found.
Dependencies	No <code>package.json</code> or lockfile; runtime has no npm dependency supply-chain surface.
CI/CD	No <code>.github</code> workflow directory in the current tree.
Data	<code>data.json</code> is public learning content and should not contain private source-only material.

Severity	Confidence	Status	Finding
Medium	9/10	Fixed	Startup failure path used dynamic innerHTML; fixed in this pass.

Performance Notes

Recent performance improvements are preserved: course data normalization is cached per course, stats rendering uses DOM nodes, TTS voice lookup is cached, and card transitions no longer rely on forced synchronous layout reads. This pass does not change the normal render hot path except for the cold error state.

Build, Run, and Test Commands

Purpose	Command
Run static site	<code>python -m http.server 8022 --bind 127.0.0.1</code>
Full tests	<code>node --test</code>
Syntax check	<code>node --check script.js</code>
Whitespace check	<code>git diff --check</code>
Browser QA URL	<code>http://127.0.0.1:8022/</code>

Verification Evidence

Check	Result
Baseline tests	<code>node --test</code> passed 27/27 before code changes.
Red test	Focused test failed with <code>renderLoadFailure</code> is not a function.
Focused test after fix	<code>node --test tests/quiz-data-normalization.test.js</code> passed 6/6.
Full tests after fix	<code>node --test</code> passed 28/28.
Syntax	<code>node --check script.js</code> exited 0.
Whitespace	<code>git diff --check</code> exited 0.
Browser normal flow	Edge loaded <code>script.js?v=22</code> , switched to basic verbs, showed 35 day options, and revealed the answer.
Browser failure flow	Forced <code>data.json 500</code> rendered the load-failure heading; injected-looking input created 0 <code>image/script</code> tags.

Remaining Risks

Risk	Status	Next Action
Manual Notion-to-data.json sync can drift.	Accepted risk.	Continue using data tests and live Pages checks after content syncs.
No committed secret-scanner config.	Accepted risk for this static personal site.	Add <code>.gitignore</code> or equivalent if secrets or automation return.
No CSP meta policy.	Accepted risk.	Consider adding and browser-testing a CSP only if the app starts handling untrusted content.

Risk	Status	Next Action
data.json is public.	Accepted by design.	Do not place private or source-only study notes in deployed data.